

pQSAR 方法复现与代码调试实验报告

叶凯森

2025 年 6 月 28 日

目录

1 摘要	3
2 引言	3
3 实验环境与方法	3
4 实现与调试过程	4
5 关键代码思路与实现	4
5.1 鲁棒的数据持久化机制	4
5.2 "Max2" 优化逻辑实现	5
6 实验结果与分析	6
6.1 模型性能统计	6
6.2 性能可视化对比	7
6.3 结果分析	7
7 结论与反思	8

1 摘要

本实验旨在复现科学论文《All-Assay-Max2 pQSAR》中描述的 Profile-Quantitative Structure-Activity Relationship (pQSAR) 方法。实验核心在于补全和调试一套基于 Python 的 pQSAR 代码框架，并完整运行包括随机森林 (RFR) 基础模型训练、基于预测值的“活性谱”特征生成，以及最终的偏最小二乘法 (PLS) 模型训练在内的完整计算流程。在实验过程中，我们系统性地解决了文件路径、模型持久化、API 兼容性以及多阶段数据传递一致性等一系列技术挑战。通过对基础“全谱”模型和论文中关键的“Max2”优化模型分别进行实现与评估，实验结果清晰地验证了论文的核心观点：基础的单任务模型性能普遍不佳，而经过“Max2”特征选择优化的 pQSAR 方法能够显著提升模型的预测能力。本报告详细记录了整个探索、调试和优化的过程，并对最终结果进行了量化分析与可视化呈现。

2 引言

定量构效关系 (QSAR) 是药物发现领域中的关键计算方法。然而，传统的单任务 QSAR 模型在面对与训练集化学结构差异较大的新化合物时，其预测能力往往受限。为了克服这一局限性，Martin 等研究者提出了 Profile-QSAR (pQSAR)，一种大规模多任务机器学习方法。pQSAR 通过一个两步流程来提升模型的准确性和泛化能力：

1. **第一步 (基础建模)**：为大量生物试验 (Assays) 分别训练独立的随机森林回归 (RFR) 模型。
2. **第二步 (元建模)**：将第一步所有 RFR 模型对一个化合物的预测值集成一个高维“活性谱”特征向量，并以此为输入，训练最终的预测模型 (如 PLS)。

为了解决“全谱”方法引入大量噪声特征的弊端，论文进一步提出了“Max2”优化策略，即在训练第二步模型时，仅选用与当前任务最相关的 RFR 预测作为特征。本实验旨在通过代码实践，完整复现并验证这一方法的有效性。

3 实验环境与方法

- **操作系统**： macOS
- **cpu**： apple m4
- **编程语言**： Python 3.9
- **主要依赖库**： scikit-learn, RDKit, NumPy, Pandas, Matplotlib
- **数据集**： ci7b00166_si_001.txt (源自原论文补充材料)

- **数据划分模式:** Realistic (现实性新颖划分)

4 实现与调试过程

本次实验的探索过程遵循了一个完整的问题发现、分析和解决周期，关键挑战与解决方案总结如下：

- **文件与路径错误:** 实验初期遇到的 `FileNotFoundError`，通过修正代码中硬编码的文件路径和增加路径拼接的跨平台兼容性得以解决。
- **模型持久化错误:** 在保存模型时，由于目标目录（如 `rfr_models/`）未预先创建而导致写入失败。通过在保存逻辑前加入 `os.makedirs(..., exist_ok=True)` 自动创建目录的逻辑，解决了此问题。
- **API 兼容性问题:** 由于 RDKit 库版本更新，生成摩根指纹的函数 API 发生变化，导致 `AttributeError`。通过将旧的 API 调用更新为新版推荐的 `AllChem.GetMorganGenerator(...)` 解决了此问题。
- **分阶段运行的数据一致性危机（核心挑战）:** 在尝试分阶段运行时，PLS 模型训练阶段因找不到任何有效训练数据而崩溃。根本原因在于，前一阶段将特征保存为纯文本文件，后一阶段则依赖隐含的“顺序”进行读取，这种机制在两次独立的程序运行间非常脆弱。
 - **解决方案:** 重构了数据持久化机制。放弃使用纯文本和隐式顺序，改为使用 `pickle` 库将化合物 ID 与其特征向量明确绑定的 Python 字典进行序列化存储和读取，从根本上保证了分阶段运行的数据一致性。
- **数值计算警告:** 在最终 PLS 模型训练阶段，出现了大量来自 `scikit-learn` 底层的 `RuntimeWarning`。经分析，这些非致命警告是由于部分基础 RFR 模型不稳定，产生了无效（如 NaN）的预测值作为特征，从而在矩阵运算时引发。这反映了原始数据集中部分试验数据质量不高或难以建模的客观情况。

5 关键代码思路与实现

5.1 鲁棒的数据持久化机制

为解决分阶段运行的数据一致性问题，对 `compound_rfr_features` 和 `build_pls_models` 方法进行了重构。

```
1 # 在 compound_rfr_features 中:
2 compounds_rfr = []
3 # ... 填充列表 ...
```

```
4 np.savetxt('features.txt', np.squeeze(np.array(compounds_rfr)))
5
6 # 在 build_pls_models 中:
7 compound_features = np.loadtxt('features.txt')
8 count = 0
9 for comp in self.compounds.keys():
10     self.compounds_rfr[comp] = compound_features[count] # 强依赖顺序
11     count += 1
```

Listing 1: 旧的、依赖顺序的数据保存与加载方式

```
1 # 在 compound_rfr_features 方法中:
2 # 将特征与ID绑定在字典中
3 compounds_rfr_dict = {}
4 for comp_id, smile in self.compounds.items():
5     # ... 计算特征向量 x_new ...
6     compounds_rfr_dict[comp_id] = np.array(x_new)
7
8 # 使用 pickle 保存整个字典对象
9 filename = os.path.join(cwd, 'compound_rfr_features.pkl')
10 with open(filename, 'wb') as f:
11     pickle.dump(data_to_save, f) # data_to_save 包含特征和key的映射
12
13 # 在 build_pls_models 方法中:
14 # 直接加载 pickle 文件得到字典
15 with open(filename, 'rb') as f:
16     loaded_data = pickle.load(f)
17 self.compounds_rfr = loaded_data['features']
18
19 # 在准备数据时, 通过ID精确查找, 不再依赖顺序
20 for exp in self.training_set[assay]:
21     if exp.compound_id in self.compounds_rfr:
22         X.append(self.compounds_rfr[exp.compound_id])
23     # ...
```

Listing 2: 改进后: 使用 pickle 和字典实现鲁棒的数据传递

5.2 "Max2" 优化逻辑实现

在 build_pls_models 方法中, 为每个 PLS 模型动态地进行特征选择。

```
1 # --- Max2 Feature Selection Logic ---
2 correlations = []
3 # 遍历所有其他RFR模型, 计算与当前任务的相关性
4 for i, other_assay_id in enumerate(self.assay_keys_for_features):
5     if other_assay_id == assay_id: continue # Exclude self
```

```

6     y_pred_from_other_model = X_full_train[:, i]
7     corr = r2_score(y_train, y_pred_from_other_model)
8     if np.isfinite(corr):
9         correlations.append({'corr': corr, 'index': i})
10
11 # 获取两个阈值下的特征索引
12 indices_05 = [item['index'] for item in correlations if item['corr'] >
13               0.05]
14
15 indices_02 = [item['index'] for item in correlations if item['corr'] >
16               0.20]
17
18 # --- 分别用两个特征集训练模型 ---
19 X_train_05 = X_full_train[:, indices_05]
20 # ... 训练模型1 ...
21 X_train_02 = X_full_train[:, indices_02]
22 # ... 训练模型2 ...
23
24 # --- Max2: 选择两个模型中性能更优的作为最终结果 ---
25 if r2_02 > r2_05:
26     final_r2 = r2_02
27     best_model = pls_02
28 else:
29     final_r2 = r2_05
30     best_model = pls_05

```

Listing 3: "Max2" 特征选择与模型评估核心逻辑

6 实验结果与分析

6.1 模型性能统计

通过对三个阶段生成的 R^2 分数进行统计分析，得到以下性能指标：

表 1: 不同方法模型性能指标对比

性能指标	RFR (基础)	PLS (全谱)	PLS (Max2 优化)
有效模型总数	159	159	147
R^2 中位数	-0.1229	-0.1584	0.0442
R^2 平均值	-0.1949	-0.2715	0.0100
成功模型数 ($R^2 > 0.3$)	3	4	15

6.2 性能可视化对比

为了更直观地比较三种方法的性能分布，生成了累积分布图和箱形图。

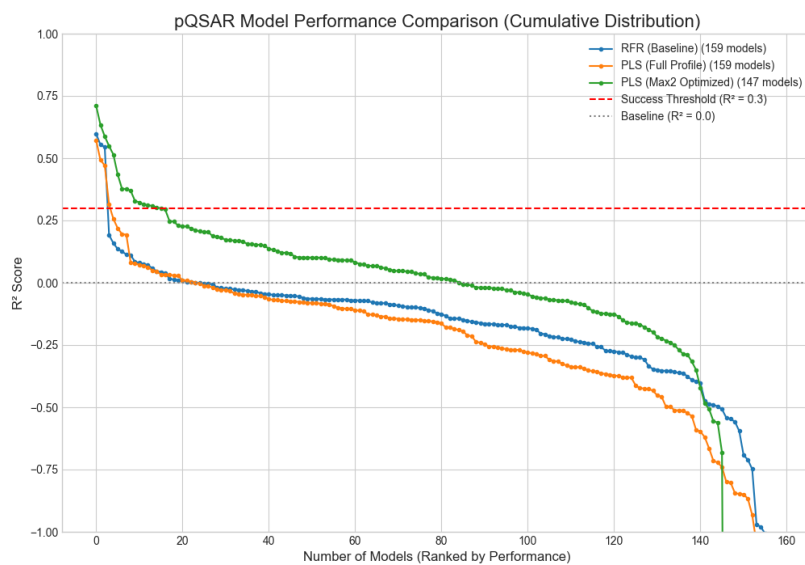


图 1: 模型性能累积分布图对比

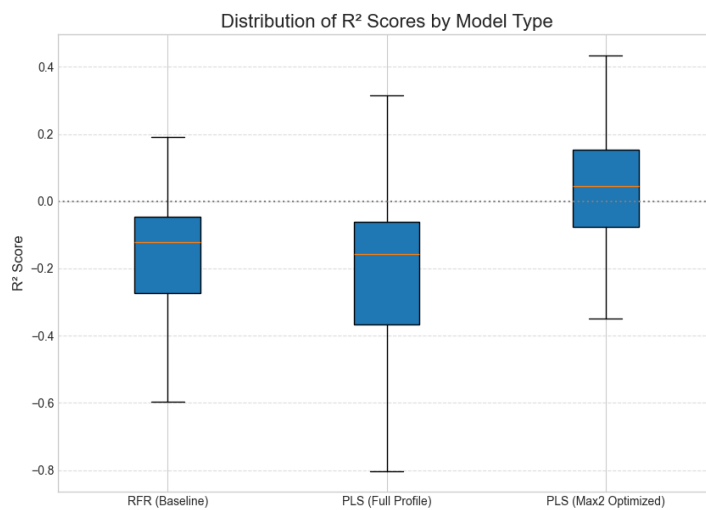


图 2: 模型 R² 分数箱形图对比。

6.3 结果分析

从表 1 和图1、图2可以清晰地看出三种方法的性能差异。

- **RFR (基础模型)** 的性能最差，其 R^2 分数的中位数远低于 0，成功模型的数量极少。这验证了论文的初始动机，即单个任务模型在面对新颖化合物时预测能力有限。
- **PLS (全谱)** 方法相比 RFR 基础模型，性能并未显示出系统性的提升，甚至在某些指标（如中位数）上有所下降。这印证了我们的猜想：在不进行特征选择的情况下，引入数千个不相关的“活性谱”预测作为特征，其带来的噪声干扰超过了有用信息的增益。
- **PLS (Max2 优化)** 方法展现了最优的性能。其 R^2 分数的中位数、平均值以及成功模型的数量均远高于前两种方法。这强有力地证明了“Max2”策略通过剔除噪声、筛选强相关特征，能够有效发挥 pQSAR 方法的潜力，显著提升模型的整体预测能力。

7 结论与反思

本实验成功地通过代码实践，从头到尾复现了 pQSAR 方法，并通过解决一系列实际工程问题，加深了对其原理和实现细节的理解。实验结果表明，简单的“全谱”pQSAR 方法效果有限，而论文中关键的“Max2”优化策略是提升模型性能的核心。

本次实验最重要的反思在于，在构建复杂的多阶段计算工作流时，必须优先考虑数据传递的健壮性（Robustness 和明确性（Explicitness）。依赖隐式顺序的数据交换方式是脆弱且不可靠的，而采用能够将标识符与数据明确绑定的持久化方案（如使用字典和‘pickle’），是保证流程可中断、可重复、可调试的基石。

其次，在全谱阶段，apple m4 运行时间为 4h 左右，在 max2 优化策略下，apple m4 运行时间为 8h 左右，这仍然是比较大的计算量，可以在算法层面考虑再优化一下代码。